

How well does SPARQL federation work in practice? A real-world analysis of query federation over large biological SPARQL endpoints

Elias Crum^{1,2}[0009–0005–3991–754X], Bryan-Elliot Tam¹, Jonni Hasinki¹,
Ana-Claudia Sima³[0000000332134495], Tarcisio Mendes de
Farias³[0000000231755372], Jerven Bolleman⁴[0000–0002–7449–1266], and Ruben
Taelman¹[0000–0001–5118–256X]

¹ IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

² Flemish institute for Technological Research (VITO) Mol, Belgium

³ SIB Swiss Institute of Bioinformatics, Lausanne, Switzerland

⁴ Swiss-Prot group, SIB Swiss Institute of Bioinformatics, Geneva, Switzerland

Abstract. Federated SPARQL querying promises integrated access to distributed knowledge graphs without requiring central data replication. However, most evaluations of SPARQL federation are conducted under controlled benchmark conditions and say relatively little about how federation behaves against public endpoints in everyday use. We present a real-world longitudinal study of federated querying over highly-used, large public biological SPARQL endpoints. Our study executes 62 federated queries obtained from users of these endpoints across more than seven public biological endpoints, and compares two families of federation strategies: a FedX/SPLendid-style baseline and modular variants implemented in the Comunica query engine. The results show that federation was fragile throughout the study period: many queries failed due to endpoint errors, timeouts, and other execution problems. The main finding is therefore not simply that one federation strategy outperformed another, but that the practical viability of public SPARQL federation was dominated by changing endpoint-side operational constraints. We frame the discussion of our findings from two perspectives, the user executing the federated queries and the endpoint maintainer who must balance resource availability and usage demands. Our findings raise concerns about the feasibility of federation engine usage in real-world scenarios, especially at the time of writing. We conclude with recommendations for evaluating SPARQL federation in the wild, for designing infrastructures better suited to real cross-endpoint biological queries, and for query execution provenance documentation for users.

Keywords: SPARQL Querying · Federated Querying · Semantic Web · SPARQL Endpoints

1 Introduction

Federated SPARQL querying offers an appealing model for domain-specific data access: independently maintained public resources can be queried together without requiring central replication. This is particularly attractive in the life sciences, where specialized knowledge is distributed across many data sources and cross-source questions are common. However, the practical success of federation depends not only on query semantics, but also on the behavior of the public endpoints involved. In real-world deployments, those endpoints are operational services with changing availability, response times, and usage constraints rather than stable benchmark components.

This distinction matters because many federated queries in biology rely on large public endpoints that serve broad user communities. For such infrastructures, temporary failures, timeouts, and other execution constraints can directly shape federated query success. Here we assess the operational effectiveness of federated querying: how well do domain-relevant, meaningful federated queries perform over public SPARQL endpoints under "ordinary" user conditions?

To answer that question, we conduct a longitudinal study of real-world federated biological queries over more than seven public SPARQL endpoints. Rather than treating endpoint behavior as fixed background noise, we study it as a central factor in federated execution.

1.1 Large Public SPARQL Endpoints

SPARQL endpoints expose RDF data through standard query interfaces, enabling remote access to graph-structured knowledge [?,?]. In the life sciences, major resources such as UniProt and the EBI RDF platform have adopted this model to support integrative access across linked biological datasets [?]. **jerven:** Remove EBI RDF platform, it's dead, SIB Semweb paper would be nice At the same time, public endpoints are shared infrastructure: they must balance openness with service stability, and prior monitoring work has shown that public SPARQL endpoints can vary substantially in availability, performance, and interoperability [?]. **jerven:** Monitoring of the monitoring is also an issue ;) that tool was down a lot during it so called evaluation period

1.2 Federated SPARQL Querying

Federated querying extends this model by allowing one query to combine data from multiple SPARQL services. At a high level, a federation engine decomposes a query into endpoint-specific subqueries and combines intermediate results into a final answer. This makes distributed biological querying possible without prior data consolidation, but it also introduces a dependency on multiple independently operated services. As a result, endpoint-side conditions can become a first-order determinant of execution outcomes.

1.3 Study Contributions

This paper studies the operational viability of federated querying over public biological SPARQL endpoints. Our work specifically addresses the following research questions:

- **RQ1:** To what extent do real-world federated biological queries execute successfully over public SPARQL endpoints?
- **RQ2:** How do results vary across federation strategies?
- **RQ3:** How do failure modes evolve over time in a real deployment setting?

By answering these questions, we aim to complement prior evaluations under controlled conditions with evidence from changing real-world endpoint behavior, and to clarify to what extent federated execution is limited by engine design versus endpoint-side operational constraints.

2 Related Work

Prior work on federated SPARQL querying has primarily focused on the design of federation algorithms that optimize query planning and intermediate result join efficiency. Crucially, the evaluation of these algorithms is performed using controlled benchmark conditions. This body of work has produced important insights into source selection, query decomposition, and distributed join processing, but it offers a less complete picture of how federation behaves against public endpoints operating under changing real-world conditions. Our work builds on that literature, but shifts the emphasis from algorithmic efficiency in stable environments to operational behavior in the wild.

2.1 Federated Querying Algorithms

SPARQL federation has been studied extensively as a problem of decomposing a query across multiple endpoints and combining intermediate results efficiently. Popular systems such as FEDX and SPLENDID illustrate two influential execution styles. FEDX performs source selection dynamically at query time, avoiding dependence on precomputed source catalogs [?], whereas SPLENDID exploits dataset metadata, in particular VoID descriptions, to guide source selection and planning [?]. Other work also explored adaptive approaches such as ANAPSID, which were designed for more volatile endpoint environments [?]. More generally, the literature treats federation as a query optimization problem under incomplete statistic availability and limited coordination between independently operated endpoints.

Recent work has also made it easier to experiment with alternative federation strategies in practice. In particular, COMUNICA provides a modular query engine architecture in which different source-selection, query planning, and join algorithm approaches can be combined and compared [?]. This modularity is

useful for our study, since it allows established federation strategies to be reimplemented and evaluated under a common execution framework. In contrast to prior work that mainly asks which strategy performs best in controlled settings, we ask how such strategies behave when exposed to unstable public endpoint conditions.

2.2 Federated Querying Benchmarks & Evaluations

Along with experimental testing of SPARQL federation approaches, benchmarks and empirical evaluations for federated SPARQL querying also exist. FEDBENCH presents one such benchmark suite for federated query processing, providing curated datasets and benchmark queries for comparing federation engines [?,?]. More recent efforts such as LARGERDFBENCH aimed to scale this style of evaluation to much larger data volumes and more demanding federation settings [?]. Other benchmark-oriented work has also questioned how well synthetic or curated workloads reflect the structure and difficulty of real query behavior, for example in the context of Wikidata-oriented SPARQL benchmarking [?].

These benchmarks are highly valuable for controlled comparison, but they typically assume relatively stable execution environments. By design, they abstract away from many of the operational characteristics of public SPARQL services, such as endpoint-side timeouts, rate limiting, transient HTTP failures, changing metadata availability, and day-to-day fluctuations in responsiveness. In real-world conditions, work has shown that public SPARQL endpoints can vary substantially in availability, interoperability, and performance [?]. This matters especially in biology, where large public SPARQL endpoints such as UniProt and the EBI RDF platform function not merely as benchmark targets, but as shared research infrastructure used by broad communities [?,?]. In such settings, federated query behavior is shaped not only by engine design, but also by endpoint operational policies and infrastructure constraints.

Our study is closest to this latter concern. Rather than introducing a new federation algorithm or benchmark suite, we examine, using two of the most popular federation techniques, how federated biological queries behave longitudinally over large public endpoints. In that sense, our study complements prior work on federation algorithms and benchmark design by focusing on operational fragility, failure modes, and the gap between benchmark expectations and everyday use.

3 Methods

1. A study of over 60 real-world federated queries obtained from users of large biological SPARQL endpoints.
2. An empirical comparison of two families of federation strategies, including FedX and SPLENDID-style approaches as well as modular variants of each implemented in Comunica.

3. Longitudinal findings that practical federation was limited in initial investigations, and that a small window of partial success in March–September 2025 was followed by complete failure after endpoint-side rate limits were introduced in fall 2025.
4. Methodological and infrastructure lessons from evaluating SPARQL federation “in the wild.”

3.1 General Study Design

3.2 Real-World Queries

Queries: Where did they come from and why did we choose to use them? Endpoints: the 7+ public biological endpoints included, selection criteria, and why they matter.

3.3 Federation Approaches

Federation approaches: what exactly counts as the two compared approaches, and what modular variations were implemented in Comunica.

3.4 Experimental Set-up

Observation period: the time span over which experiments were rerun, and why a longitudinal design matters.

Other key design decisions: Execution environment Timeout settings Retry policy Number of runs per query/configuration Logging and error capture Definition of success/failure Whether you assess only execution success or also result stability/completeness

3.5 Measurement Metrics

Metrics that matter: query success rate, runtime / timeout rate, endpoint error rate, number/type of HTTP failures, result reproducibility over time, differences across federation strategies

4 Results

4.1 Overall picture

Federation never worked robustly across the full workload. Many queries failed throughout the study due to endpoint errors, timeouts, or other operational issues. Improvements from engine-side changes were limited.

4.2 Experimental Phase 1: limited federation viability

(February – Sept 2025) Present the queries that reliably produced results.

Characterize what made them different: fewer endpoints, lighter intermediate results, more selective joins, more stable sources, or simpler source selection.

Show that limited success was possible, but only for a narrow subset of the workload.

This subsection should make the reader think: “perhaps federation can work in practice, but only under constrained conditions.”

4.3 Experimental Phase 2: endpoint-side rate limiting becomes decisive

(Sept 2025 – Present) Show that previously stable queries stopped succeeding. Document the failure pattern tied to rate limiting.

Emphasize that the major change was not a query-engine regression, but a change in the operational environment.

jerven: During this period LLMs came into the picture, and with them significant new abusive non robots respecting crawlers.

4.4 Comparison across federation approaches

Which approach did better before fall 2025? Did modular Comunica adjustments help for the few viable queries? Were any gains washed out once endpoint-side limits tightened? The framing here should be: engine differences mattered at the margins -> endpoint-side constraints dominated the final outcome

4.5 Failure taxonomy

A concise taxonomy represented in this git repo:

timeouts endpoint unavailability HTTP/rate-limit errors unstable or inconsistent responses planning/source-selection inefficiencies failures caused by contacting too many sources or issuing too many subqueries This subsection turns messy empirical observations into a reusable scientific result.

5 Discussion

Our findings suggest that the question is not only whether federation engines can optimize distributed queries, but whether public endpoint infrastructures are willing and able to support the access patterns that federation requires.

Prelim themes:

The main bottleneck was operational, not purely algorithmic. Public endpoints may be suitable for interactive single-endpoint querying, but not necessarily for cross-endpoint federated workloads. Longitudinal instability matters: a result observed in one month may not reproduce a few months later. Benchmark-based federation evaluation may overestimate practical viability. Federation over public infrastructure may require a different deployment model: negotiated access, mirrors, caches, precomputed summaries, or hybrid architectures.

6 Limitations

One important move here is to present temporal variability not only as a limitation, but as part of the phenomenon being studied.

biological domain specificity only 7+ endpoints only $\tilde{60}$ queries real-world queries may not cover all federation patterns changing endpoint behavior makes causal attribution difficult results may depend on timeout thresholds and experimental infrastructure

7 Practical Implications and Recommendations

Recommendations can be directed to three audiences: Researchers: evaluate federation longitudinally, not only on static benchmarks. Engine designers: incorporate rate-limit-aware planning, request budgeting, caching, and degraded-execution strategies. Endpoint operators / infrastructure designers: expose clearer operational policies and consider supported federation access modes.

Acknowledgments. Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N) and (Long Research Stay V416635N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).